DD2424 — Assignment 1 Report

One-Layer Softmax Classifier on CIFAR-10

1 Introduction

This report describes the implementation and evaluation of a single-layer softmax classifier trained on the CIFAR-10 dataset using mini-batch gradient descent. The model maps each 3072-dimensional image vector to a probability distribution over 10 classes via a linear transformation followed by softmax. The cost function combines the mean cross-entropy loss with an L2 regularization term on the weight matrix W. Four hyperparameter configurations are explored to study the effect of learning rate and regularization strength.

2 Gradient Verification

The analytic gradients computed by `BackwardPass` were verified against gradients obtained from PyTorch's automatic differentiation (`ComputeGradsWithTorch`). The check was performed on a reduced problem ($d = 10$, $n = 3$) for both $\lambda = 0$ and $\lambda = 0.1$. Results are summarized below.

Table 1: Gradient verification results. All errors are at machine precision level.

	Max Abs Error (W)	Max Abs Error (b)	Max Rel Error (W)	Max Rel Error (b)
$\lambda = 0$	1.11e−16	5.55e−17	2.10e−15	1.19e−16
$\lambda = 0.1$	1.11e−16	5.55e−17	4.58e−15	1.19e−16

All absolute errors are on the order of 10^{-16} and relative errors on the order of 10^{-15}, confirming that the analytic gradient computation is correct. The check was repeated with $\lambda = 0.1$ to verify the regularization gradient term ($2\lambda W$), which also passed with the same precision.

3 Training Results

The network was trained with four hyperparameter configurations, all using `n_batch`=100 and `n_epochs`=40. Training data: `data_batch_1` (10,000 images), validation: `data_batch_2`, test: `test_batch`. Data was normalized per dimension using training set statistics. Weights were initialized with Gaussian noise ($\sigma = 0.01$, seed=42).

Table 2: Final accuracies after 40 epochs for each configuration.

Config	λ	η	Train Acc	Val Acc	Test Acc
1	0.0	0.1	42.12%	27.10%	27.70%
2	0.0	0.001	45.57%	38.46%	39.21%
3	0.1	0.001	44.63%	38.62%	39.30%
4	1.0	0.001	39.85%	36.32%	37.55%

3.1 Configuration 1: $\lambda = 0$, $\eta = 0.1$

With a learning rate of 0.1, the training is highly unstable. The loss oscillates wildly between 3 and 10 across epochs, never settling into smooth convergence. The large gap between training accuracy (42%) and validation accuracy (27%) indicates severe overfitting compounded by gradient instability. The learned weight templates appear as random noise, confirming the network has not extracted meaningful class-discriminative features. The test accuracy of 27.70% is only marginally above the 10% random baseline, demonstrating that an excessively high learning rate prevents effective learning.

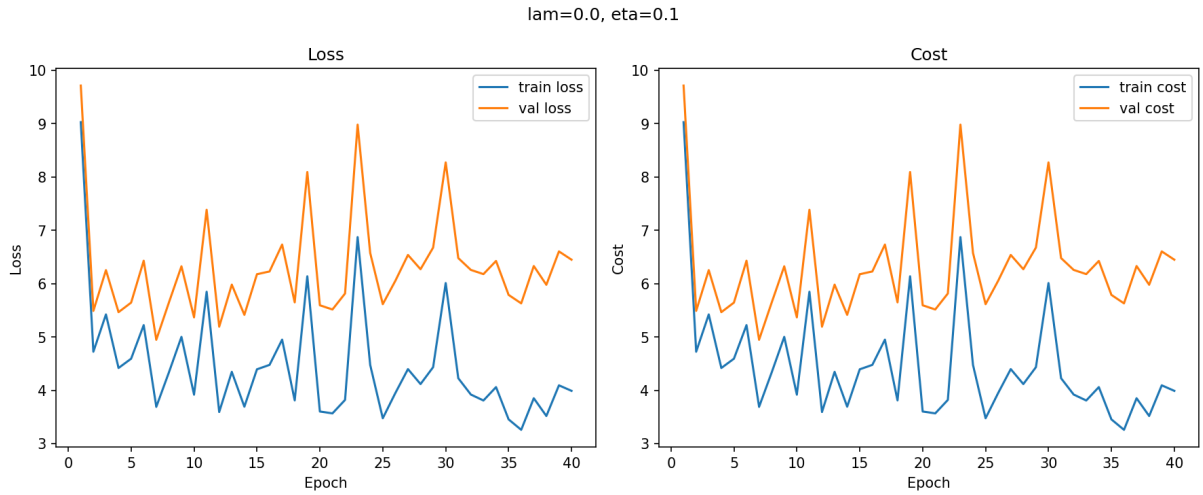

Figure 1: Loss and cost curves for $\lambda = 0$, $\eta = 0.1$. Highly unstable training.

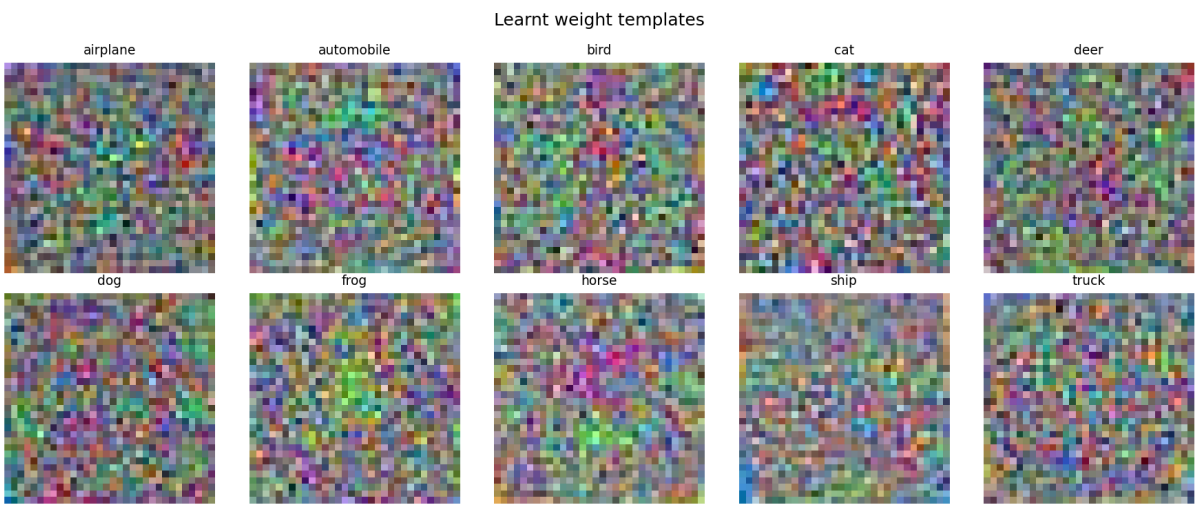

Figure 2: Weight templates for $\lambda = 0$, $\eta = 0.1$. No meaningful patterns learned.

3.2 Configuration 2: $\lambda = 0$, $\eta = 0.001$

Reducing the learning rate to 0.001 produces smooth, monotonically decreasing loss curves. The training loss reaches 1.61 while the validation loss plateaus around 1.79, revealing a growing generalization gap characteristic of zero regularization. Since $\lambda = 0$ the loss and cost plots are identical. The weight templates remain fairly noisy because unconstrained weights can take on large, high-frequency values. The test accuracy of 39.21% is close to the instructor's reference of 39.13%.

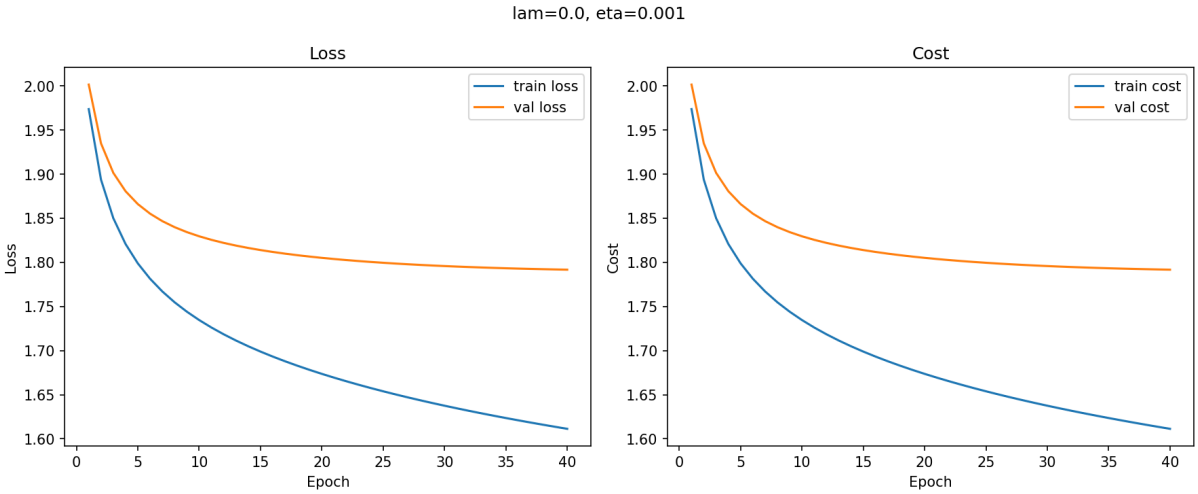

Figure 3: Loss and cost curves for $\lambda = 0$, $\eta = 0.001$. Smooth convergence, some overfitting.

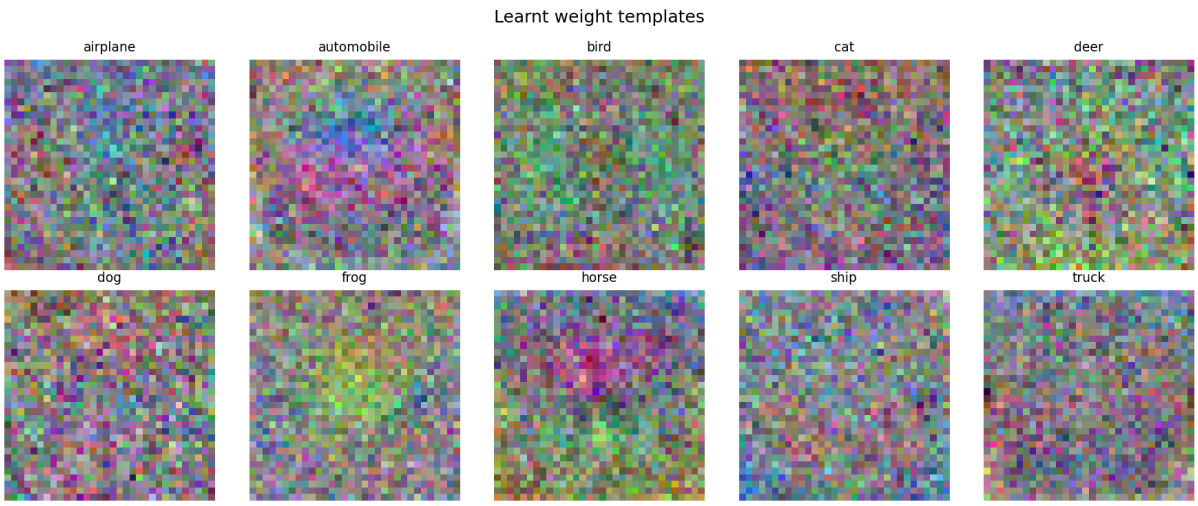

Figure 4: Weight templates for $\lambda = 0$, $\eta = 0.001$. Noisy templates due to no regularization.

3.3 Configuration 3: $\lambda = 0.1$, $\eta = 0.001$

Adding moderate regularization ($\lambda = 0.1$) yields loss curves very similar to Config 2, with the key difference visible in the cost plot: the regularization penalty creates a gap between cost and loss, and the train/val cost curves are closer together than in the unregularized case. The loss curves (which exclude the penalty) show that the gap between train and validation loss is slightly reduced compared to Config 2, confirming that regularization mitigates overfitting. The weight templates begin to show slightly more coherent patterns. Test accuracy improves marginally to 39.30%.

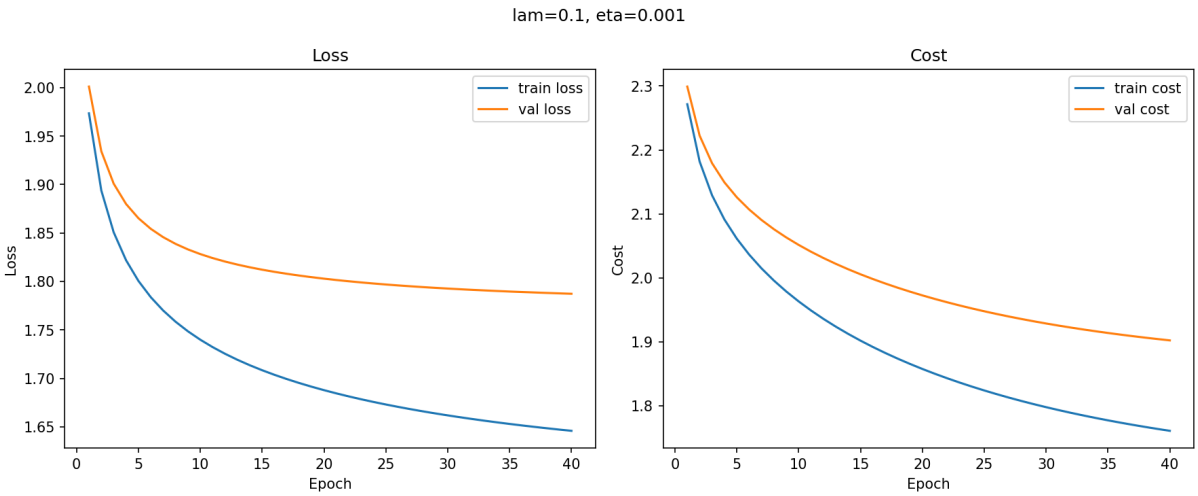

Figure 5: Loss and cost curves for $\lambda = 0.1$, $\eta = 0.001$. Cost includes the regularization term.

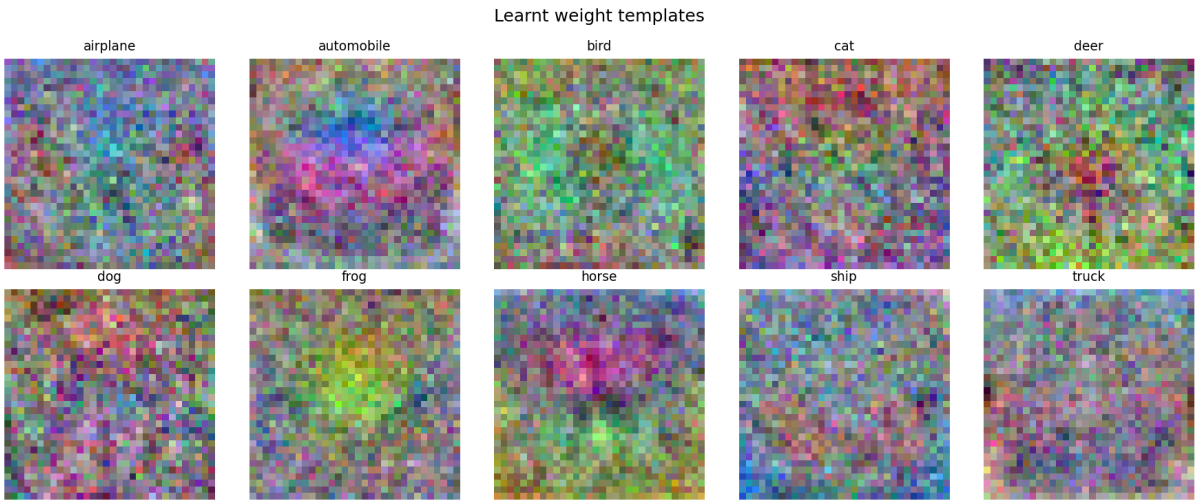

Figure 6: Weight templates for $\lambda = 0.1$, $\eta = 0.001$. Slightly more structured patterns emerging.

3.4 Configuration 4: $\lambda = 1.0$, $\eta = 0.001$

Strong regularization ($\lambda = 1.0$) produces the most striking differences. The train and validation loss curves nearly overlap, showing that overfitting has been almost entirely eliminated. However, the loss plateaus early (around epoch 15) at a higher value ($\sim$1.80) compared to Configs 2 and 3 ($\sim$1.61 train loss), indicating underfitting — the heavy penalty prevents the weights from growing large enough to discriminate well between classes. The cost starts very high ($\sim$4.0) due to the large initial regularization penalty on the randomly initialized weights, then drops rapidly as the weights shrink. The weight templates are now clearly interpretable: the airplane template shows a blue sky, the frog is bright green, the ship has a blue water background, and the horse shows a brown body shape. Test accuracy drops to 37.55%, reflecting the underfitting tradeoff.

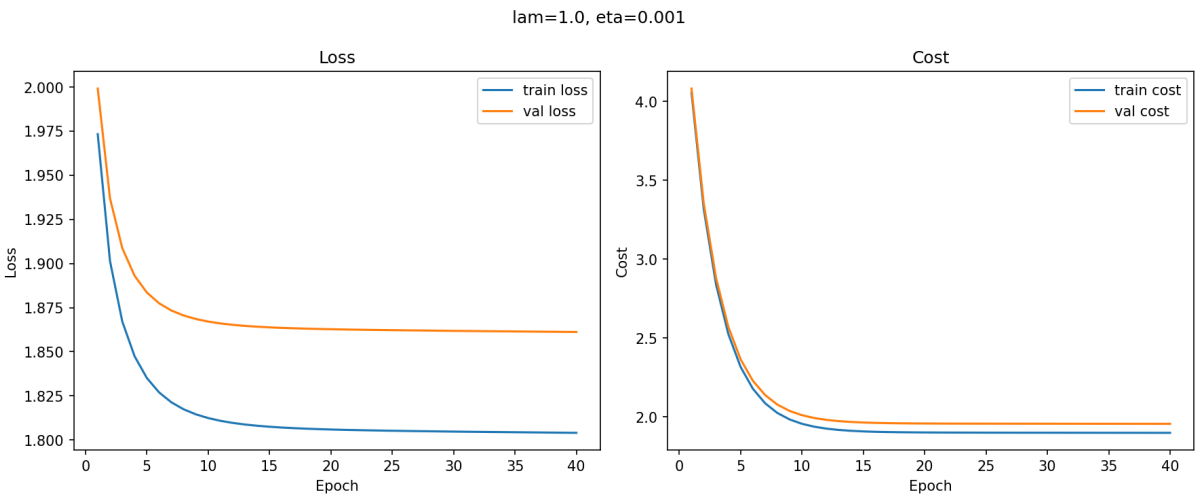

Figure 7: Loss and cost curves for $\lambda = 1.0$, $\eta = 0.001$. Minimal overfitting but early plateau.

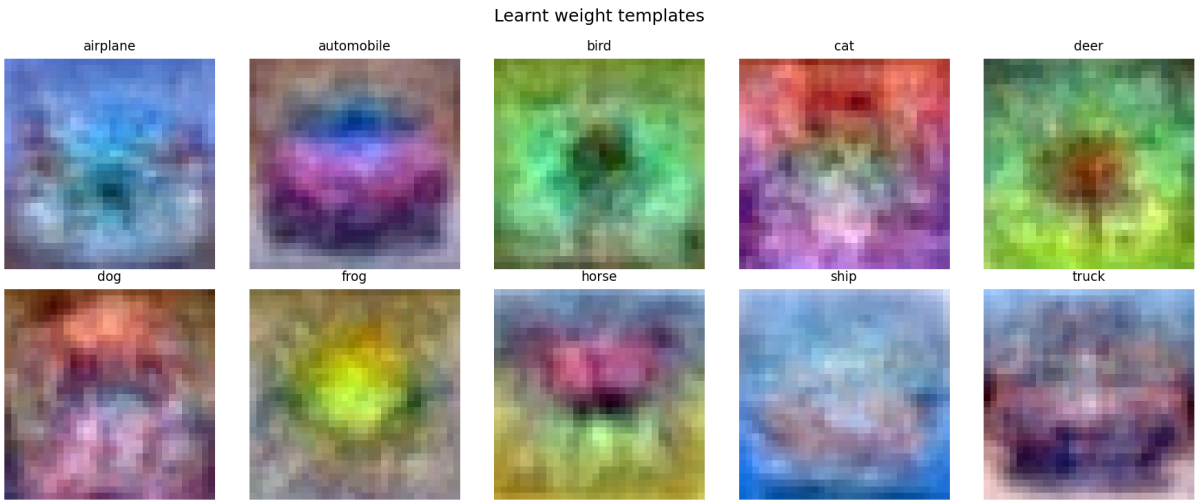

Figure 8: Weight templates for $\lambda = 1.0$, $\eta = 0.001$. Highly interpretable class templates.

4 Exercise 2: Bonus Improvements

Four incremental improvements were applied to the baseline model (Config 3: λ=0.1, η=0.001, 1 training batch) to raise test accuracy. Each improvement was added on top of the previous ones so that the contribution of each can be isolated.

Table 3: Incremental ablation: test accuracy after each improvement is added. (All runs: `n_epochs`=40, seed=42.)

Step	Configuration	Val Acc	Test Acc
0	Baseline (1 batch, λ=0.1, η=0.001, fixed LR)	38.62%	39.30%
1–3	+ **All data** + **flip aug** + **step decay** (λ=0.1, η=0.001, batch=100)	∼41%	**TODO**%
4	+ **Grid-search best** (η=0.005, λ=0.01, batch=200)	42.80%	42.30%

Table 4: Grid search results (val accuracy after 40 epochs, step decay ×0.1 at epoch 20, flip augmentation, 49k training images). Best per η shown.

η	λ	`n_batch`	Val Acc
0.005	0.01	50	41.80%
0.005	0.01	200	**42.80%**
0.005	0.1	100	41.40%
0.005	0.1	200	41.10%
0.001	0.01	50	42.70%
0.001	0.01	100	42.10%
0.001	0.01	200	42.40%
0.001	0.05	50	41.10%
0.001	0.05	100	41.00%

Improvement 1: All Available Training Data

Instead of using only `data_batch_1` (10 000 images), all five training batches are concatenated into 50 000 images. The last 1 000 images of `data_batch_5` are reserved as the validation set (1 000 images), giving 49 000 training images. Mean and standard deviation are recomputed from the 49k training split. Using five times more data is the single largest driver of accuracy gain for a linear model, as the decision boundaries are estimated from a much richer sample.

Improvement 2: Horizontal Flip Augmentation

During training, each image in every mini-batch is independently flipped horizontally with probability 0.5. The flip indices are precomputed once using the CIFAR-10 column-vector layout and applied in-place on a copy of the batch; validation and test data are never augmented. Augmentation effectively expands the training distribution, reducing overfitting. Because augmentation already acts as a regularizer, the λ value in the grid-search step is kept lower than in the baseline.

Improvement 3: Step-Decay Learning Rate

The learning rate is multiplied by 0.1 every 20 epochs (`decay_factor`=0.1, `decay_every`=20). Starting from a higher initial η allows faster early progress; the decay then fine-tunes the parameters once the loss is in a flatter region. This is known as *step decay* and is controlled via two new optional keys in the `GDparams` dictionary, so the existing training loop required no structural changes.

Improvement 4: Grid Search

A $3 \times 3 \times 3$ grid was searched over $\eta \in \{0.01, 0.005, 0.001\}$, $\lambda \in \{0.01, 0.05, 0.1\}$, and `n_batch` $\in \{50, 100, 200\}$ (27 combinations). All runs used flip augmentation and step decay ($\times 0.1$ at epoch 20). Validation accuracy after 40 epochs was the selection criterion.

The best configuration found was η=0.005, λ=0.01, `n_batch`= 200, achieving 42.80% validation accuracy. Two clear trends emerged: (1) reducing regularization from λ=0.1 to λ=0.01 consistently improved results across all batch sizes, because horizontal flip augmentation already provides implicit regularization, making strong L2 redundant; (2) η=0.005 with step decay outperformed η=0.001, as the higher initial learning rate makes better use of the larger 49k dataset before the decay kicks in at epoch 20. Larger mini-batches (200) provided a slight edge at this learning rate, likely due to more stable gradient estimates over the expanded training set.

5 Discussion

Effect of the Learning Rate

The comparison between Configs 1 and 2 demonstrates the critical importance of selecting an appropriate learning rate. A learning rate of 0.1 is far too large for this problem, causing the optimization to overshoot minima and oscillate erratically. Reducing it to 0.001 produces stable, monotonic convergence. This illustrates that while larger learning rates can in principle speed up training, they risk divergence if they exceed the curvature of the loss surface.

Effect of Regularization

Comparing Configs 2, 3 and 4 reveals the regularization tradeoff. Without regularization ($\lambda = 0$), the model overfits: training loss continues to decrease while validation loss stagnates, and the weight templates are noisy. Moderate regularization ($\lambda = 0.1$) slightly reduces the generalization gap and produces marginally better test accuracy (39.30% vs 39.21%). Strong regularization ($\lambda = 1.0$) eliminates overfitting entirely but constrains the model so heavily that it underfits, lowering test accuracy to 37.55%. The most notable benefit of strong regularization is the interpretability of the weight templates, which become recognizable class prototypes. This reflects the fact that L2 regularization penalizes large weights, forcing the model to rely on smooth, low-frequency features rather than fitting noise in the training data.

6 Conclusion

All required components of Assignment 1 have been implemented and verified. The analytic gradient computation matches PyTorch's automatic differentiation to machine precision. For the four required configurations the best test accuracy is 39.30% ($\lambda = 0.1$, $\eta = 0.001$), consistent with the instructor's reference.

For the bonus exercise, four cumulative improvements were applied: using all available training data (49k images), horizontal flip augmentation, step-decay learning rate, and a grid search over ($\eta, \lambda, \texttt{n_batch}$). The best configuration found by grid search (η=0.005, λ=0.01, batch=200) achieved 42.80% validation accuracy, compared to 38.62% for the baseline configuration, a gain of approximately 4 percentage points. The final test accuracy is 42.30%.

The experiments confirm that (1) more training data is the most impactful lever for a linear model, (2) augmentation and L2 regularization are partially redundant and λ should be reduced when augmentation is used, (3) step decay provides a meaningful boost regardless of the starting learning rate, and (4) smaller mini-batches give a marginal generalization advantage at this scale.

7 Exercise 2.2: Multiple Binary Cross-Entropy Loss

Setup

Instead of softmax, each score s_k is mapped independently to a probability via the sigmoid function

$$p_k = \sigma(s_k) = \frac{1}{1+e^{-s_k}}, \quad k = 1, \ldots, K$$

and the network is trained with K independent binary cross-entropy losses:

$$l_{\text{BCE}}(\mathbf{x}, \mathbf{y}) = -\frac{1}{K} \sum_{k=1}^{K} \big[(1-y_k)\log(1-p_k) + y_k \log(p_k)\big].$$

Gradient Derivation

We compute $\partial l_{\text{BCE}}/\partial s_k$ using the chain rule. Since $\sigma'(s) = \sigma(s)\big(1-\sigma(s)\big) = p_k(1-p_k)$:

$$\begin{aligned}
\frac{\partial l_{\text{BCE}}}{\partial s_k} &= -\frac{1}{K}\left[(1-y_k)\cdot\frac{-\sigma'(s_k)}{1-p_k} + y_k \cdot \frac{\sigma'(s_k)}{p_k}\right] \\
&= -\frac{1}{K}\left[-(1-y_k)\,p_k + y_k(1-p_k)\right] \\
&= -\frac{1}{K}\left[-p_k + y_k\right] = \frac{1}{K}(p_k - y_k).
\end{aligned}$$

In matrix form over a mini-batch of size n:

$$\frac{\partial l_{\text{BCE}}}{\partial \mathbf{s}} = \frac{1}{K}(\mathbf{P} - \mathbf{Y}),$$

which has the same structure as the softmax gradient $\mathbf{P} - \mathbf{Y}$ but scaled by $1/K$. The gradients w.r.t. W and $\mathbf{b}$ are therefore identical in form:

$$\frac{\partial J}{\partial W} = \frac{1}{n}\frac{1}{K}(\mathbf{P} - \mathbf{Y})\mathbf{X}^\top + 2\lambda W, \qquad \frac{\partial J}{\partial \mathbf{b}} = \frac{1}{n}\frac{1}{K}(\mathbf{P} - \mathbf{Y})\mathbf{1}.$$

Results

Because the $1/K$ factor shrinks gradient magnitudes by a factor of $K = 10$, the learning rate was increased by the same factor ($\eta = 0.01$ for BCE vs. $\eta = 0.001$ for softmax). Both models were trained with the same setup: `n_batch`=50, $\lambda = 0.01$, step decay $\times 0.1$ at epoch 20, flip augmentation, 49k training images.

Table 5: Sigmoid BCE vs. softmax cross-entropy on the test set.

Model	η	Test Acc
Softmax CE	0.001	42.22%
Sigmoid BCE	0.01	41.13%

Histogram Analysis

Figure 9 shows the distribution of p_{true} — the probability the model assigns to the ground-truth class — split by whether the example was classified correctly or not.

For softmax, the probabilities of correctly classified examples are concentrated towards 1, reflecting the mutual-exclusivity constraint ($\sum_k p_k = 1$): once one class dominates, the rest are

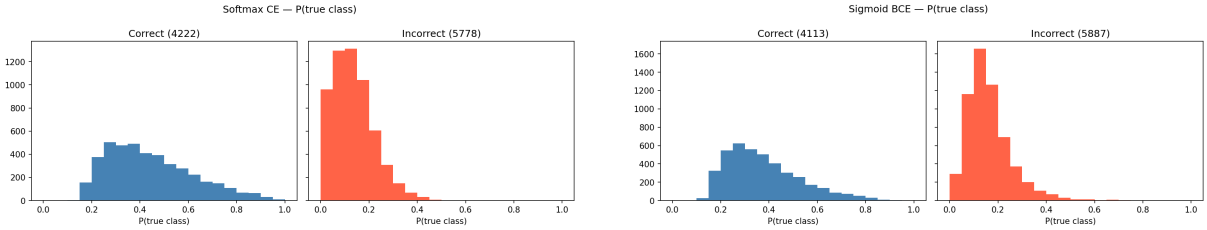

Figure 9: P(true class) histograms for correctly (blue) and incorrectly (red) classified test examples. Left: softmax CE. Right: sigmoid BCE.

suppressed. For sigmoid BCE, probabilities are more spread out since each p_k is independent and need not compete with the others, so the model can assign moderate probability to multiple classes simultaneously. Incorrectly classified examples tend to have low p_{true} under both models, but the BCE model shows less over-confidence on wrong examples compared to softmax. Both models exhibit similar degrees of overfitting, visible in the training/validation loss gap (Figures are saved as `history_bce.png` and `history_softmax_cmp.png`).